

Experiment 'p_6_6_alternating_sum_array' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_6_alternating_sum_array

Natural language statement: Compute the alternating sum of all elements in an array. For example, if your method reads the input [1, 4, 9, 16, 9, 7, 4, 9, 11] then it computes $1 - 4 + 9 - 16 + 9 - 7 + 4 - 9 + 11 = -2$.

Method signature: p_6_6_alternating_sum_array(inputs: array<int>) returns (alt_sum: int)

Ensures

- `alt_sum == sum_array(inputs[..], 1)`

Functional Code Given

```
function sum_array(inputs: seq<int>, i: int): int
{
  if |inputs| == 0 then 0
  else inputs[0] * i + sum_array(inputs[1..], i * -1)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Compute the alternating sum of all elements in an array. For example, if your method
→ reads the input [1, 4, 9, 16, 9, 7, 4, 9, 11] then it computes $1 - 4 + 9 - 16 + 9 - 7 + 4 - 9 + 11 = -2$.

The signature should be:

`p_6_6_alternating_sum_array(inputs: array<int>) returns (alt_sum: int)`

The method should respect the following contract:

`ensures alt_sum == sum_array(inputs[..], 1)`

The contract uses the following dafny code:

```
function sum_array(inputs: seq<int>, i: int): int
{
  if |inputs| == 0 then 0
  else inputs[0] * i + sum_array(inputs[1..], i * -1)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn1cLJLQzqfJV7BNsc31EsMjekSXe

Model: gpt-5.5-2026-04-23

Created at: 1780576661

Finish reason: stop

Usage: 552 tokens in, and 6467 tokens out

```
{
  ghost var a := inputs[..];
  alt_sum := 0;
  var i := 0;
  var sign := 1;
  assert a[0..] == a;
  while i < inputs.Length
    invariant 0 <= i <= inputs.Length
    invariant |a| == inputs.Length
```

```

invariant a == inputs[..]
invariant alt_sum + sum_array(a[i..], sign) == sum_array(a, 1)
decreases inputs.Length - i
{
  var oldI := i;
  var oldSign := sign;
  var oldAlt := alt_sum;
  var x := inputs[oldI];

  assert x == a[oldI];
  assert |a[oldI..]| > 0;
  assert (a[oldI..])[0] == a[oldI];
  assert (a[oldI..])[1..] == a[oldI + 1..];
  assert sum_array(a[oldI..], oldSign) == x * oldSign + sum_array(a[oldI + 1..], oldSign
    ↪ * -1);
  assert oldAlt + x * oldSign + sum_array(a[oldI + 1..], oldSign * -1) == sum_array(a,
    ↪ 1);

  alt_sum := oldAlt + x * oldSign;
  sign := oldSign * -1;
  i := oldI + 1;

  assert alt_sum + sum_array(a[i..], sign) == sum_array(a, 1);
}
assert i == inputs.Length;
assert i == |a|;
assert a[i..] == [];
assert sum_array(a[i..], sign) == 0;
assert alt_sum == sum_array(a, 1);
assert a == inputs[..];
assert sum_array(a, 1) == sum_array(inputs[..], 1);
}

```

Final Program

```

function sum_array(inputs: seq<int>, i: int): int
{
  if |inputs| == 0 then 0
  else inputs[0] * i + sum_array(inputs[1..], i * -1)
}

method p_6_6_alternating_sum_array(inputs: array<int>) returns (alt_sum: int)
  ensures alt_sum == sum_array(inputs[..], 1)
{
  ghost var a := inputs[..];
  alt_sum := 0;
  var i := 0;
  var sign := 1;
  assert a[0..] == a;
  while i < inputs.Length

```

```

invariant 0 <= i <= inputs.Length
invariant |a| == inputs.Length
invariant a == inputs[..]
invariant alt_sum + sum_array(a[i..], sign) == sum_array(a, 1)
decreases inputs.Length - i
{
  var oldI := i;
  var oldSign := sign;
  var oldAlt := alt_sum;
  var x := inputs[oldI];

  assert x == a[oldI];
  assert |a[oldI..]| > 0;
  assert (a[oldI..])[0] == a[oldI];
  assert (a[oldI..])[1..] == a[oldI + 1..];
  assert sum_array(a[oldI..], oldSign) == x * oldSign + sum_array(a[oldI + 1..], oldSign
    ↪ * -1);
  assert oldAlt + x * oldSign + sum_array(a[oldI + 1..], oldSign * -1) == sum_array(a,
    ↪ 1);

  alt_sum := oldAlt + x * oldSign;
  sign := oldSign * -1;
  i := oldI + 1;

  assert alt_sum + sum_array(a[i..], sign) == sum_array(a, 1);
}
assert i == inputs.Length;
assert i == |a|;
assert a[i..] == [];
assert sum_array(a[i..], sign) == 0;
assert alt_sum == sum_array(a, 1);
assert a == inputs[..];
assert sum_array(a, 1) == sum_array(inputs[..], 1);
}

```

Total Token Usage

Input tokens: 552

Output tokens: 6467

Reasoning tokens: 5948

Sum of 'total tokens': 7019

Experiment Timings

Overall Experiment started at 1780576658827, ended at 1780576810650, lasting 151823ms (151.82 seconds)

Iteration #1 started at 1780576659005, ended at 1780576810650, lasting 151645ms (151.65 seconds)